

LAPORAN TUGAS 4

SISTEM OPERASI

Dosen pengampu : Triawan Adi Cahyanto M.Kom



Disusun oleh :

NELSAN HADI SYAPUTRA

(2410651017)

UNIVERSITAS MUHAMMADIYAH JEMBER

FAKULTAS TEKNIK

PROGRAM STUDY TEKNIK INFORMATIKA

2025

1. TUGAS 1 : EKSPERIMEN CPU SHEDULING DENGAN DATA YANG BERBEDA

- **DATA SET 1 :**

First Come First Served :

```
nelsan@NELSAN:~$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 3

Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 1
  Burst Time: 5

Proses P3
  Arrival Time: 2
  Burst Time: 8

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
P1      0    10    10     10     0
P2      1     5    15     14     9
P3      2     8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== GANTT CHART ===
| P1 | P2 | P3 |
0   10  15  23
nelsan@NELSAN:~$
```

Shorted Job First :

```
nelsan@NELSAN:~$ ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 3
Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 1
  Burst Time: 5

Proses P3
  Arrival Time: 2
  Burst Time: 8

=== HASIL PENJADWALAN SJF ===
PID    AT    BT    CT    TAT    WT
P1      0    10    10     10     0
P2      1     5    15     14     9
P3      2     8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== URUTAN EKSEKUSI ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P3
```

- **DATA SET 2 :**

First Come First Served :

```

nelsan@NELSAN:~$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 4

Proses P1
  Arrival Time: 0
  Burst Time: 5

Proses P2
  Arrival Time: 1
  Burst Time: 3

Proses P3
  Arrival Time: 2
  Burst Time: 8

Proses P4
  Arrival Time: 3
  Burst Time: 6

=== HASIL PENJADWALAN FCFS ===
PID   AT   BT   CT   TAT   WT
P1     0    5    5     5    0
P2     1    3    8     7    4
P3     2    8   16    14    6
P4     3    6   22    19   13

Average Turnaround Time: 11.25
Average Waiting Time: 5.75

=== GANTT CHART ===
| P1 | P2 | P3 | P4 |
0   5   8  16  22

```

Shorted Job First :

```

nelsan@NELSAN:~$ ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 4
Proses P1
  Arrival Time: 0
  Burst Time: 5

Proses P2
  Arrival Time: 1
  Burst Time: 3

Proses P3
  Arrival Time: 2
  Burst Time: 8

Proses P4
  Arrival Time: 3
  Burst Time: 6

=== HASIL PENJADWALAN SJF ===
PID   AT   BT   CT   TAT   WT
P1     0    5    5     5    0
P2     1    3    8     7    4
P3     2    8   22    20   12
P4     3    6   14    11    5

Average Turnaround Time: 10.75
Average Waiting Time: 5.25

=== URUTAN EKSEKUSI ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P4 -> P3
nelsan@NELSAN:~$

```

2. TUGAS 2: ANALISIS EKSPERIMEN KETIGA ALGORITMA

Jawab pertanyaan berikut:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
2. Mengapa hasil algoritma berbeda-beda?
3. Kapan sebaiknya menggunakan algoritma FCFS?
4. Kapan sebaiknya menggunakan algoritma SJF?
5. Kapan sebaiknya menggunakan algoritma Round Robin?

Jawaban :

1. Algoritma mana yang memberikan Average Waiting Time terkecil?

Biasanya, SJF (Shortest Job First) memberikan Average Waiting Time (AWT) paling kecil.

Karena SJF selalu mengeksekusi proses dengan Burst Time (BT) paling pendek terlebih

dahulu, sehingga proses kecil tidak menunggu lama, dan total waktu tunggu rata-rata menjadi lebih efisien.

Contoh :

FCFS → AWT cukup tinggi karena proses pertama selalu dijalankan dulu, meskipun lama.

SJF → AWT paling kecil.

RR → AWT menengah, tergantung nilai quantum.

2. Mengapa hasil algoritma berbeda-beda?

Karena setiap algoritma memiliki strategi penjadwalan CPU yang berbeda, yaitu:

- FCFS (First Come First Served):urut berdasarkan waktu kedatangan proses.
→ Proses pertama datang akan dieksekusi dulu, tanpa mempertimbangkan lamanya burst time.
- SJF (Shortest Job First):urut berdasarkan burst time terpendek.
→ Fokus efisiensi waktu tunggu, tapi bisa menyebabkan *starvation* (proses panjang menunggu terus).

- Round Robin: setiap proses mendapat jatah waktu (*time quantum*) bergiliran.
→ Cocok untuk sistem multitasking dan adil untuk semua proses, meski efisiensinya sedikit berkurang.

3. Kapan sebaiknya menggunakan algoritma FCFS?

Gunakan **FCFS** ketika:

- Semua proses memiliki waktu burst yang hampir sama.
- Sistem sederhana (misal batch system atau sistem antrian non-interaktif).
- Tidak ada kebutuhan interaktif yang ketat.

4. Kapan sebaiknya menggunakan algoritma SJF?

Gunakan SJF ketika:

- Proses dan burst time sudah diketahui sebelumnya (misalnya pada sistem batch).
- Tujuan utama adalah meminimalkan waktu tunggu dan waktu penyelesaian rata-rata.
- Tidak ada banyak variasi waktu kedatangan proses baru.

5. Kapan sebaiknya menggunakan algoritma Round Robin?

Gunakan Round Robin ketika:

- Sistem bersifat interaktif atau time-sharing.
- Setiap proses perlu mendapat kesempatan CPU yang adil.
- Respons cepat untuk setiap proses lebih penting daripada efisiensi total waktu tunggu.

3. TUGAS 3: MODIFIKASI PROGRAM

Tambahkan fitur berikut pada salah satu program:

1. Tampilkan Gantt Chart yang lebih detail
2. Simpan hasil ke file text
3. Baca input dari file

- Tambahan code source Gantt Chart

```
// Gantt Chart Detail
printf("\n=== GANTT CHART ===\n");
printf(" ");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < proc[i].burst_time; j++) printf("--");
    printf(" ");
}
printf("\n|");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < proc[i].burst_time - 1; j++) printf(" ");
    printf("P%d", proc[i].pid);
    for (int j = 0; j < proc[i].burst_time - 1; j++) printf(" ");
    printf("|");
}
printf("\n ");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < proc[i].burst_time; j++) printf("--");
    printf(" ");
}
printf("\n0");
for (int i = 0; i < n; i++) {
    int space = proc[i].burst_time * 2;
    printf("%*d", space, proc[i].completion_time);
}
printf("\n");
```

- Tambahan code source untuk menyimpan hasil ke file text

```
// Simpan hasil ke file output
fprintf(fout, "=== HASIL PENJADWALAN FCFS ===\n");
fprintf(fout, "PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    fprintf(fout, "P%d\t%d\t%d\t%d\t%d\t%d\n",
        proc[i].pid, proc[i].arrival_time, proc[i].burst_time,
        proc[i].completion_time, proc[i].turnaround_time, proc[i].waiting_time);
}
fprintf(fout, "\nRata-rata TAT : %.2f", total_tat / n);
fprintf(fout, "\nRata-rata WT : %.2f\n", total_wt / n);
fprintf(fout, "\n=== GANTT CHART ===\n");
for (int i = 0; i < n; i++) fprintf(fout, "| P%d ", proc[i].pid);
fprintf(fout, "|\n0");
for (int i = 0; i < n; i++) fprintf(fout, "    %d", proc[i].completion_time);
fprintf(fout, "\n");

fclose(fout);
printf("\nHasil juga disimpan ke file 'hasil_fcfs.txt'\n");
```

- Tambahan code source untuk membaca intup dari file

```
FILE *fin, *fout;

// Membuka file input
fin = fopen("input_fcfs.txt", "r");
if (fin == NULL) {
    printf("File input_fcfs.txt tidak ditemukan!\n");
    return 1;
}

// Membuka file output untuk menyimpan hasil
fout = fopen("hasil_fcfs.txt", "w");
if (fout == NULL) {
    printf("Gagal membuat file hasil_fcfs.txt!\n");
    fclose(fin);
    return 1;
}

// Membaca jumlah proses dari file input
fscanf(fin, "%d", &n);
struct Process proc[n];

// Membaca data proses dari file
for (int i = 0; i < n; i++) {
    proc[i].pid = i + 1;
    fscanf(fin, "%d %d", &proc[i].arrival_time, &proc[i].burst_time);
}

fclose(fin);
```

- Compile dan buat file input_fcfs.txt, masukkan file sebagai berikut :

```
3
0      10
1      5
2      8
```

- Maka hasil outputnya akan seperti ini :

```
nelson@NELSAN:~$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: ^Z
[1]+  Stopped                  ./fcfs
nelson@NELSAN:~$ nano fcfs.c
nelson@NELSAN:~$ gcc fcfs.c -o fcfs
nelson@NELSAN:~$ nano input.txt
nelson@NELSAN:~$ ./fcfs
File input_fcfs.txt tidak ditemukan!
nelson@NELSAN:~$ mv input.txt input_fcfs.txt
nelson@NELSAN:~$ ./fcfs
=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
P1      0    10    10    10     0
P2      1     5    15    14     9
P3      2     8    23    21    13

Rata-rata TAT : 15.00
Rata-rata WT  : 7.33

=== GANTT CHART ===
-----
|          P1          |      P2      |      P3      |
-----
0          10        15          23

Hasil juga disimpan ke file 'hasil_fcfs.txt'
nelson@NELSAN:~$
```

4. Tugas 4: Eksperimen dengan data berbeda untuk perbandingan algoritma

- Test Case 1: Proses dengan burst time bervariasi :

```

nelson@NELSAN:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 4
Masukkan time quantum: 3
Proses P1
  Arrival Time: 0
  Burst Time: 24

Proses P2
  Arrival Time: 1
  Burst Time: 3

Proses P3
  Arrival Time: 2
  Burst Time: 3

Proses P4
  Arrival Time: 3
  Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 21
Waktu 3: Menjalankan P1 -> Sisa waktu: 18
Waktu 6: Menjalankan P2 -> Selesai pada waktu 9
Waktu 9: Menjalankan P3 -> Selesai pada waktu 12
Waktu 12: Menjalankan P4 -> Sisa waktu: 3
Waktu 15: Menjalankan P1 -> Sisa waktu: 15
Waktu 18: Menjalankan P1 -> Sisa waktu: 12
Waktu 21: Menjalankan P1 -> Sisa waktu: 9
Waktu 24: Menjalankan P4 -> Selesai pada waktu 27
Waktu 27: Menjalankan P4 -> Selesai pada waktu 27

=== HASIL PENJADWALAN ROUND ROBIN ===
PID    AT    BT    CT    TAT    WT
P1      0    24    -1141240173    0    0
P2      1     3     9     8     5
P3      2     3    12    10     7
P4      3     6    27    24    18

Average Turnaround Time: 10.50
Average Waiting Time: 7.50
nelson@NELSAN:~$

```

- Test Case 2: Proses datang bersamaan :

```

nelson@NELSAN:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 5
Masukkan time quantum: 4
Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 0
  Burst Time: 5

Proses P3
  Arrival Time: 0
  Burst Time: 8

Proses P4
  Arrival Time: 0
  Burst Time: 12

Proses P5
  Arrival Time: 0
  Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 6
Waktu 4: Menjalankan P2 -> Sisa waktu: 1
Waktu 8: Menjalankan P3 -> Sisa waktu: 4
Waktu 12: Menjalankan P4 -> Sisa waktu: 8
Waktu 16: Menjalankan P5 -> Sisa waktu: 2
Waktu 20: Menjalankan P1 -> Sisa waktu: 2
Waktu 24: Menjalankan P1 -> Selesai pada waktu 26
Waktu 26: Menjalankan P2 -> Selesai pada waktu 27
Waktu 27: Menjalankan P2 -> Selesai pada waktu 27
Waktu 27: Menjalankan P3 -> Selesai pada waktu 31
Waktu 31: Menjalankan P3 -> Selesai pada waktu 31

=== HASIL PENJADWALAN ROUND ROBIN ===
PID    AT    BT    CT    TAT    WT
P1      0    10    26    26    16
P2      0     5    27    27    22
P3      0     8    31    31    23
P4      0    12   32767  -228584144   32767
P5      0     6   2102215258    0     0

Average Turnaround Time: -45716812.00
Average Waiting Time: 6565.60
nelson@NELSAN:~$

```

- Test Case 3: Proses datang dengan interval

```

nlsan@NELSAN:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 5
Masukkan time quantum: 2
Proses P1
  Arrival Time: 0
  Burst Time: 8
Proses P2
  Arrival Time: 2
  Burst Time: 4
Proses P3
  Arrival Time: 4
  Burst Time: 9
Proses P4
  Arrival Time: 6
  Burst Time: 5
Proses P5
  Arrival Time: 8
  Burst Time: 3

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 6
Waktu 2: Menjalankan P1 -> Sisa waktu: 4
Waktu 4: Menjalankan P2 -> Sisa waktu: 2
Waktu 6: Menjalankan P1 -> Sisa waktu: 2
Waktu 8: Menjalankan P1 -> Selesai pada waktu 10
Waktu 10: Menjalankan P3 -> Sisa waktu: 7
Waktu 12: Menjalankan P1 -> Selesai pada waktu 12
Waktu 12: Menjalankan P2 -> Selesai pada waktu 14
Waktu 14: Menjalankan P4 -> Sisa waktu: 3
Waktu 16: Menjalankan P2 -> Selesai pada waktu 16
Waktu 16: Menjalankan P1 -> Selesai pada waktu 16

=== HASIL PENJADWALAN ROUND ROBIN ===
PID   AT   BT   CT   TAT   WT
P1     0    8   16   16    8
P2     2    4   16   14   10
P3     4    9  499126280  32575  498783904
P4     6    5   32767  -564731984  32767
P5     8    3  499122778    0    0

Average Turnaround Time: -112939880.00
Average Waiting Time: 99763344.00

```

5. Tugas 5. Analisis Eksperimen perbandingan algoritma

❖ Analisis Performa

a. Dalam kondisi apa FCFS memberikan hasil terbaik?

- FCFS terbaik ketika **urutan kedatangan sudah “baik”** — artinya proses datang dalam urutan yang kira-kira meningkat atau burst time relatif homogen (jarang ada proses sangat panjang di depan proses pendek).

- Cocok juga untuk **sistem batch** di mana overhead context switch harus minimal dan fairness sederhana cukup.
- Singkat: FCFS bagus bila variasi *burst time* kecil dan tidak perlu respons interaktif.

b. Mengapa SJF hampir selalu memberikan Average Waiting Time terkecil?

- Karena SJF memilih proses dengan *burst time* terkecil terlebih dahulu, sehingga banyak proses pendek cepat selesai dan tidak menunggu lama — ini mengurangi jumlah total waktu tunggu.
- Secara teori (untuk instance statis di mana semua BT diketahui) SJF **meminimalkan average waiting time** dibandingkan urutan lainnya.
- Catatan: SJF mengoptimalkan *rata-rata*, bukan fairness; proses panjang bisa terpengaruh.

c. Apa trade-off dari Round Robin?

- **Kelebihan:** adil, bagus untuk interactive/time-sharing, memberikan respons periodik ke semua proses.
- **Kekurangan:** jika quantum terlalu kecil → banyak context switch (overhead tinggi) → throughput turun; jika quantum terlalu besar → mendekati FCFS (respons buruk untuk proses interaktif).
- Jadi trade-off: *responsivitas / fairness vs overhead / throughput*.

❖ **Context Switching**

a. Algoritma mana yang paling banyak melakukan context switch?

- **Round Robin** (terutama dengan quantum kecil) melakukan context switch paling banyak, karena CPU berpindah antar proses setiap quantum.
- **SJF non-preemptive** dan **FCFS** melakukan sedikit context switch (hanya saat proses selesai atau saat CPU idle menunggu proses baru).
- **SRTF (preemptive SJF)** jika dipakai bisa melakukan context switch cukup sering (ketika proses baru dengan remaining time lebih kecil tiba).

b. Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?

- Quantum ↓ (lebih kecil) → jumlah context switch ↑ (lebih sering berpindah), overhead ↑.
- Quantum ↑ (lebih besar) → jumlah context switch ↓, perilaku mendekati FCFS/non-preemptive, responsivitas ↓.
- Ada trade-off optimal: quantum harus cukup besar supaya overhead switching kecil dibanding eksekusi aktual, tapi cukup kecil agar respons cepat.

3. Coba jalankan RR dengan quantum 2, 4, 8, 16 — apa yang terjadi?

Umum yang terjadi ketika kamu coba pada satu test set:

- **q = 2:** Context switch sangat sering → Gantt chart banyak segmen; average waiting time cenderung tinggi karena overhead; latency per proses kecil (respons awal cepat).
- **q = 4:** Lebih baik dari q=2: switching berkurang, respons masih baik.
- **q = 8:** Switching lebih sedikit; untuk banyak kasus mendekati efisiensi yang baik — latency sedikit lebih tinggi, overhead sedang.
- **q = 16:** Hampir seperti FCFS untuk proses yang burst < 16 akan selesai tanpa preemption; switching sedikit; responsiveness menurun jika banyak proses pendek.
Kesimpulan praktis: quantum ~ beberapa kali rata-rata waktu layanan pendek sering jadi pilihan baik. Uji empiris pada datasetmu untuk menemukan nilai “manis”nya.

❖ Fairness

a. Algoritma mana yang paling "adil" untuk semua proses?

- **Round Robin** umumnya paling adil karena setiap proses mendapat giliran CPU secara periodik (equal time slices).
- Jika fairness diukur sebagai “setiap proses mendapat kesempatan CPU dalam batas waktu wajar”, RR menang.

b. Proses mana yang paling dirugikan di algoritma FCFS?

- **Proses pendek yang datang setelah proses sangat panjang** (convoy effect). Jadi proses pendek yang datang terlambat bisa menunggu lama — mereka yang paling dirugikan di FCFS.

c. Apakah ada kemungkinan starvation di SJF?

- **Ya**, terutama pada versi **preemptive SJF (SRTF)** jika proses pendek terus datang sehingga proses panjang terus ditunda → proses panjang bisa mengalami starvation.
- Pada **SJF non-preemptive** starvation lebih kecil kemungkinan tapi masih bisa terjadi jika antrian pendek terus mendominasi kedatangan.
- Praktik nyata: mekanisme aging atau prioritas dinamis diperlukan untuk mencegah starvation.

❖ Aplikasi Dunia Nyata (Rekomendasi)

a. Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?

- Preferensi: pendekatan yang memprioritaskan request kecil → SJF-like scheduling (jika ukuran/estimasi pekerjaan tersedia) atau multilevel feedback queue (MFG) yang memberi prioritas ke pekerjaan pendek/interaktif.
- Kenyataan: web server modern pakai load balancing + event-driven IO, bukan hanya scheduler CPU klasik, tetapi prinsip “layani request kecil lebih cepat” tetap berlaku.

b. Untuk render video (task besar), algoritma mana yang cocok?

- Untuk job batch besar: FCFS (untuk kesederhanaan dan throughput) atau SJF jika durasi jobs diketahui dan tujuan meminimalkan total waktu tunggu batch.
- Biasanya: pekerja render dikelompokkan di queue terpisah (batch queue) — fairness kurang penting, throughput penting.

c. Untuk OS desktop (interactive), algoritma mana yang cocok?

- Round Robin (atau variasinya: Multilevel Feedback Queue) — sebab butuh responsivitas cepat ke proses interaktif (UI, input user) dan fairness antar aplikasi.
- OS modern gunakan MLFQ yang pada dasarnya adalah RR + prioritas dinamis untuk responsivitas dan efisiensi.